

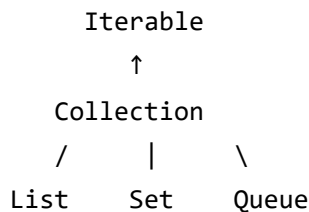
Class Note: Collection Interface and Its Methods (Java Util Package)

1. Introduction

- The **Collection Interface** is part of the **Java Collections Framework (JCF)** in the `java.util` package.
- It is the **root interface** for all collection classes like `List`, `Set`, and `Queue`.
- It defines the **basic operations** that can be performed on groups of objects, known as **elements**.

```
import java.util.Collection;
```

2. Hierarchy



- **Subinterfaces of Collection:**
 - **List** → e.g. `ArrayList`, `LinkedList`
 - **Set** → e.g. `HashSet`, `LinkedHashSet`, `TreeSet`
 - **Queue** → e.g. `PriorityQueue`, `LinkedList`

3. Key Characteristics

- Stores a **group of objects (elements)**.
- Allows **adding**, **removing**, and **iterating** over elements.
- Can **check size**, **emptiness**, and **membership**.
- Does **not** store elements in key-value pairs (that's for **Map** interface).

4. Commonly Used Methods of Collection Interface

Method	Return Type	Description / Example
<code>boolean add(E e)</code>	boolean	Adds element <code>e</code> to the collection. ✅ <code>collection.add("Apple");</code>
<code>boolean addAll(Collection<? extends E> c)</code>	boolean	Adds all elements from another collection <code>c</code> .
<code>void clear()</code>	void	Removes all elements from the collection.
<code>boolean contains(Object o)</code>	boolean	Checks if element <code>o</code> exists. ✅ <code>collection.contains("Apple");</code>
<code>boolean containsAll(Collection<?> c)</code>	boolean	Checks if collection contains all elements in <code>c</code> .
<code>boolean isEmpty()</code>	boolean	Returns true if collection has no elements.
<code>Iterator<E> iterator()</code>	Iterator	Returns an iterator to traverse the collection.
<code>boolean remove(Object o)</code>	boolean	Removes a single instance of specified element <code>o</code> .
<code>boolean removeAll(Collection<?> c)</code>	boolean	Removes all elements that are also in <code>c</code> .
<code>boolean retainAll(Collection<?> c)</code>	boolean	Keeps only the elements that are also in <code>c</code> .
<code>int size()</code>	int	Returns the number of elements in the collection.
<code>Object[] toArray()</code>	Object[]	Converts collection to an array.
<code><T> T[] toArray(T[] a)</code>	T[]	Converts collection to an array of specified type.
<code>default Stream<E> stream()</code>	Stream	Returns a sequential stream (Java 8+).

Method	Return Type	Description / Example
default Stream<E> parallelStream()	Stream	Returns a parallel stream (Java 8+).

5. Example Usage

```
import java.util.*;

public class CollectionExample {
    public static void main(String[] args) {
        Collection<String> fruits = new ArrayList<>();

        // Adding elements
        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Mango");

        // Display elements
        System.out.println("Fruits: " + fruits);

        // Check if contains an element
        System.out.println("Contains Mango? " + fruits.contains("Mango"));

        // Remove an element
        fruits.remove("Banana");
        System.out.println("After removal: " + fruits);

        // Iterate over elements
        for (String fruit : fruits) {
            System.out.println(fruit);
        }

        // Size of collection
        System.out.println("Total fruits: " + fruits.size());
    }
}
```

Output:

```
Fruits: [Apple, Banana, Mango]
Contains Mango? true
After removal: [Apple, Mango]
Apple
Mango
Total fruits: 2
```

6. Summary

- The **Collection interface** provides the **foundation** for all collection classes.
- Defines **common operations** such as adding, removing, checking membership, and iteration.
- Specialized interfaces (`List` , `Set` , `Queue`) extend it to provide additional features.